

# Compliance, mapped once

How Onam evaluates a control once and reports it against 70+ frameworks — and connects every gap to a priced attack path.

One rule → many controls

Rules as code

70+ frameworks

Custom frameworks

Gap → priced path

## EXECUTIVE SUMMARY

# Evaluate the control once. Report it against every framework.

Compliance work is mostly duplication: the same control, re-checked and re-evidenced for PCI, then SOC 2, then ISO, then a customer questionnaire. Onam removes the duplication by decoupling the check from the framework. Rules are evaluated once as code; a single finding maps to many controls across 70+ frameworks at once.

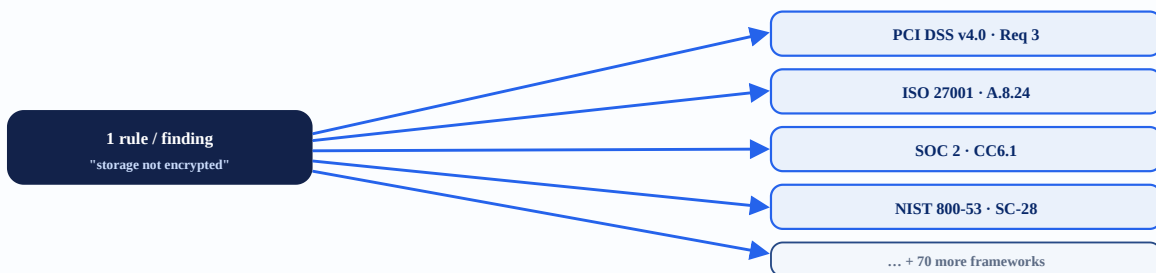
This paper explains the mapping model, why rules-as-code makes framework coverage extend automatically, and how compliance and attack-path reasoning reinforce each other — so a control gap is not just a failed check but a node on a priced path.

**An honest framing.** Framework *count* is a coverage feature, not a differentiator on its own — several platforms publish large numbers. Onam's distinct value is the *architecture* that produces the mapping (one rule → many controls, auto-extending) and the ability to connect a compliance gap to a priced attack path. This paper leads with the mechanism, not the number.

## 01 One finding, many controls

A framework is, at bottom, a set of controls. Many controls across many frameworks ask for the same underlying thing — "encrypt data at rest," "restrict public access," "enforce MFA." Onam evaluates that underlying condition once and maps the result to every control it satisfies or violates.

FIGURE 1 — THE MAPPING FAN-OUT



*One evaluated condition satisfies (or fails) many controls at once. Fix it once; the status updates everywhere it is mapped. Control IDs are illustrative of the mapping shape.*

## 02 Rules as code — why coverage extends itself

Onam's rules are versioned YAML the platform loads and evaluates; nothing is hardcoded. This is what makes framework coverage compound rather than accumulate by hand. When a new rule is added, it automatically extends every framework whose controls it maps to. When a new framework is onboarded, its controls map to existing rule IDs — no new checks to write.

- 1 Add a rule.** A new YAML rule ships; every mapped framework gains coverage for the condition it checks.
- 2 Add a framework.** A new framework's controls map to existing rule IDs; coverage appears without new detection code.
- 3 Add a custom framework.** A Rule Builder and custom-framework mapping let a team model an internal standard against the same rule library.

**The compounding effect.** Because checks and frameworks are decoupled, work done for one obligation pays down others automatically. A rule written to satisfy an internal standard may also close a PCI and an ISO control the same day — without anyone re-authoring the check.

## 03 Frameworks supported

Onam supports 70+ frameworks out of the box across regulatory, industry, and best-practice standards — for example PCI DSS v4.0, SOC 2, ISO 27001/27017, HIPAA/HITRUST, GDPR, NIST 800-53, and CIS benchmarks — with custom frameworks mappable to rule IDs. Coverage is delivered as continuous evaluation, not a point-in-time audit export.

## 04 Compliance meets the attack path

A failed control is a compliance fact. The same failing condition is also a node in the property graph — which means Onam can tell you not just *that* a control failed, but whether its failure lies on a verified, priced route to a crown jewel. That reframes remediation: two "failures" of equal audit weight are not equal if one sits on a choke point and the other reaches nothing.

FIGURE 2 — THE SAME GAP, TWO LENSES

### COMPLIANCE LENS

Control X: FAIL

*one row in an audit report*

### RISK LENS

Same gap = choke point on a \$X.XM path

*fix this one first — illustrative figure*

*Compliance says what is out of policy; the graph says which out-of-policy items an attacker can actually exploit — and prices them. Same finding, two questions answered.*

**Audit and risk from one evaluation.** The auditor gets continuous control status; the security team gets the same gaps ranked by priced exposure. Neither needs a separate scan, because both read the same findings store.

## 05 What framework count does and does not prove

**Coverage breadth is necessary, not sufficient.** A large framework number tells a buyer they will not be missing a standard they need; it does not by itself prove depth of mapping or usefulness of the result. Onam presents 70+ frameworks honestly as broad out-of-the-box coverage, and points to the mechanism — one rule to many controls, auto-extending, connected to priced paths — as the reason the coverage is maintainable and actionable rather than a static checklist. Buyers comparing vendors should weigh mapping architecture and risk-linkage, not the headline count alone.

## 06 Summary

Onam evaluates security conditions once as versioned rules and maps each result to many controls across 70+ frameworks, so a single fix updates compliance status everywhere it applies. Rules-as-code makes coverage compound: new rules extend every mapped framework, and new frameworks map to existing rules without new detection code. Because the same findings feed the property graph, a control gap can be seen as what it really is — a node that may sit on a priced attack path — letting teams satisfy auditors and reduce real risk from one evaluation. The framework count is the floor; the mapping architecture and risk-linkage are the value.

**Map your obligations once.** A read-only connect shows your current control status across every mapped framework, with the gaps that sit on priced paths flagged first.